

```
<!--Diplomado Infraestructura en TI - Modulo 4-->
```

```
PROYECTO 4: Falco + Network  
Policies (Runtime Security) {
```

```
<Integrantes:"Ricardo Juarez Ugalde"/>  
<          "Enrique Yahir Uriarte Ortiz"/>
```

```
}
```



Contenidos

- 01 Introducción
- 02 Objetivo
- 03 Arquitectura
- 04 ¿Por qué Cilium + Hubble?
- 05 Falco: ¿qué monitorea y cómo?
- 06 Reglas custom (21 reglas)
- 07 Falcosidekick: enrutamiento de alertas
- 08 Network Policies: diseño de 3 capas
- 09 Mapeo a ISO/IEC 27001
- 10 Conclusiones

Introducción {

En Kubernetes, las Network Policies y las políticas de RBAC controlan **quién puede hacer qué**, pero no detectan comportamiento anómalo una vez que un contenedor ya está corriendo

Ese es el vacío que cubre la seguridad en tiempo de ejecución (**runtime security**)

Mecanismos complementarios, no redundantes:

- **Falco:** detección de comportamiento malicioso a nivel de **syscall**, dentro del contenedor
- **Network Policies:** contención del movimiento lateral, a nivel de red, entre contenedores

}

Objetivos {

- Desplegar **Falco** como `DaemonSet` en todos los nodos del clúster
- Enrutar alertas con **Falcosidekick**, con integración a Slack y dashboard
- Implementar mínimo 15 reglas custom de Falco
- Diseñar Network Policies con:
 - `default-deny` por namespace
 - Whitelists documentadas (ingress/egress)
 - Diagrama de flujos de tráfico
 - Testing de conectividad
- Evidenciar 5 controles de **ISO/IEC 27001**
- Demostracion de implementaciones.

}

Arquitectura {

Clúster Kubernetes (kubeadm, 3 nodos)
master (control-plane) · worker1 · worker2

CNI: Cilium + Hubble

Falco (DaemonSet, 1 por nodo) → Falcosidekick → Slack + Falcosidekick UI

Namespaces de demo con Network Policies:
[frontend] → [backend] → [database]

- Clúster kubeadm de 3 nodos, Rocky Linux 9.7, containerd como runtime
- Falco corre en todos los nodos (visibilidad completa, no solo en los nodos con carga de trabajo)
- Falcosidekick centraliza y enruta las alertas a dos destinos en paralelo
- El escenario de prueba usa 3 namespaces aislados por Network Policies, simulando una arquitectura de 3 capas real

}

¿Por qué Cilium + Hubble? {

Cilium + Hubble

- Network Policies, visibilidad del tráfico en tiempo real
- Mapa de tráfico en tiempo real, con veredicto visual (`forwarded`/`dropped`) por flujo
- Se pueden ver las políticas funcionando y no solo por pruebas de conectividad (`curl`, `wget`)
- Cambio de CNI no requirió tocar ni una sola Network Policy, son manifiestos `NetworkPolicy` estándar de Kubernetes, no específicos de un CNI
- Cilium y Hubble se instalan a nivel de infraestructura del clúster, **fuera del repositorio del proyecto**

}

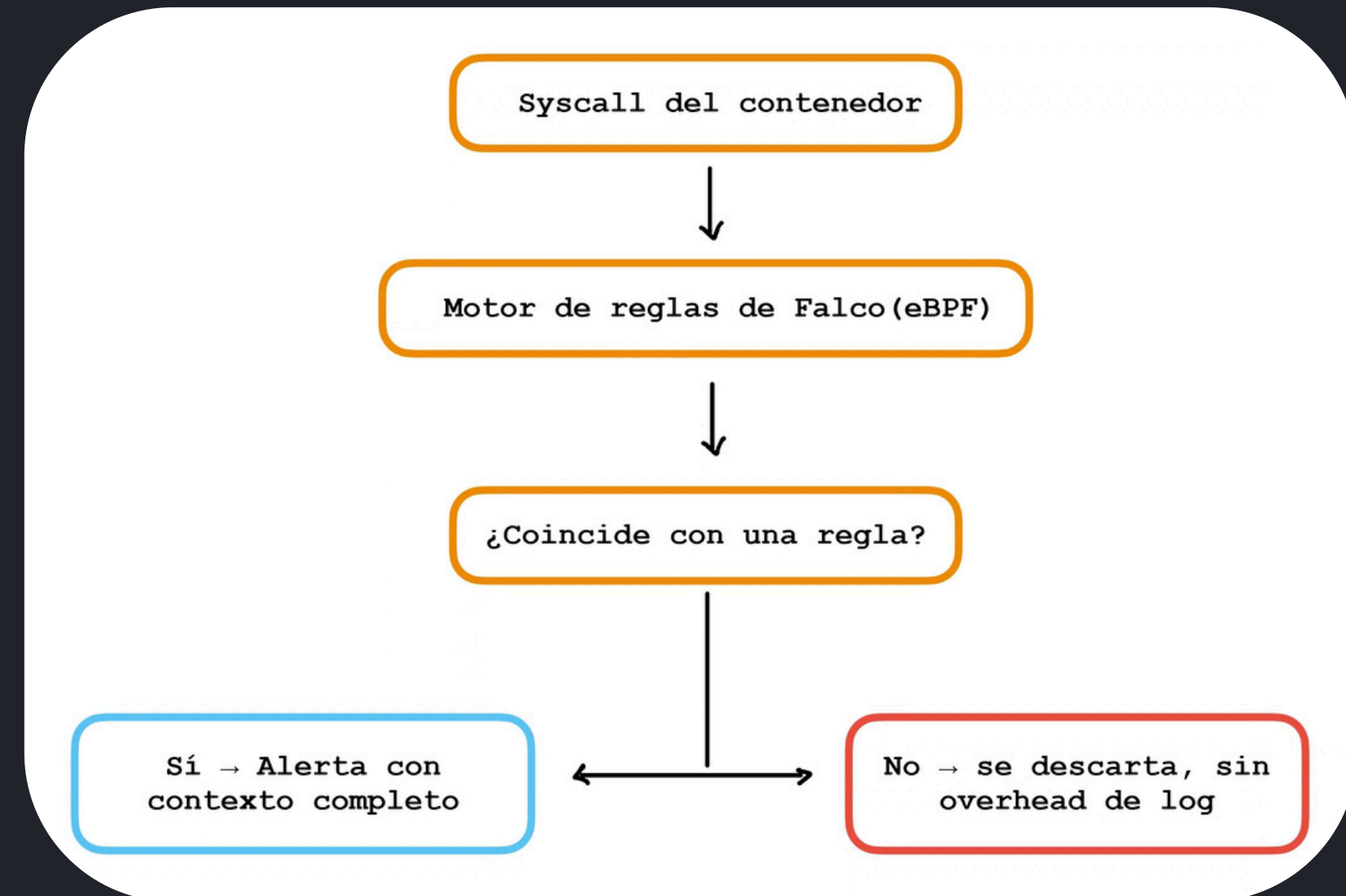
Falco: ¿qué monitorea y cómo? {



Falco intercepta llamadas al sistema (syscalls) de todos los contenedores del clúster, en tiempo real, directo desde el kernel

Cada syscall se compara contra un conjunto de **reglas**: condiciones lógicas que describen comportamiento sospechoso o prohibido

Cuando una regla coincide, Falco genera una **alerta** con contexto completo: pod, namespace, contenedor, usuario, comando ejecutado, imagen



}

Reglas custom (21 reglas) {

Grupo 1 – Acceso y ejecución en el contenedor (4 reglas)

01

Shell
spawneada en
contenedor

02

Herramienta de
reconocimiento
ejecutada

03

Herramienta de
transferencia de
red sospechosa

04

Ejecución de binario
desde directorio
temporal

Grupo 2 – Escalación de privilegios (6 reglas)

05

Uso de sudo o
su en
contenedor

07

Contenedor
privilegiado
iniciado

09

Manipulación de
namespaces de
Linux

06

Cambio
inesperado de
UID a root

08

Uso de capability
peligrosa dentro
de contenedor

10

Acceso a filesystem
del host o socket
del runtime

}

Grupo 3 – Persistencia (5 reglas)

11

Escritura en
directorio
del sistema

13

Modificación de
binario del sistema
(permisos/rename)

15

Gestión de usuarios
dentro de un contenedor

12

Modificación de
binario crítico
(escritura)

14

Modificación de
configuración de
cron

Grupo 4 – Acceso a credenciales / secrets (2 reglas)

16

Lectura de
archivo de
credenciales

17

Acceso a token de
ServiceAccount

}

Grupo 5 – Red y exfiltración/impacto (4 reglas)

18

Conexión
saliente a
puerto
inusual

19

Conexión hacia
IP en lista de
vigilancia

20

Conexión a
puerto típico de
minería de
criptomonedas

21

Posible borrado de
logs del sistema

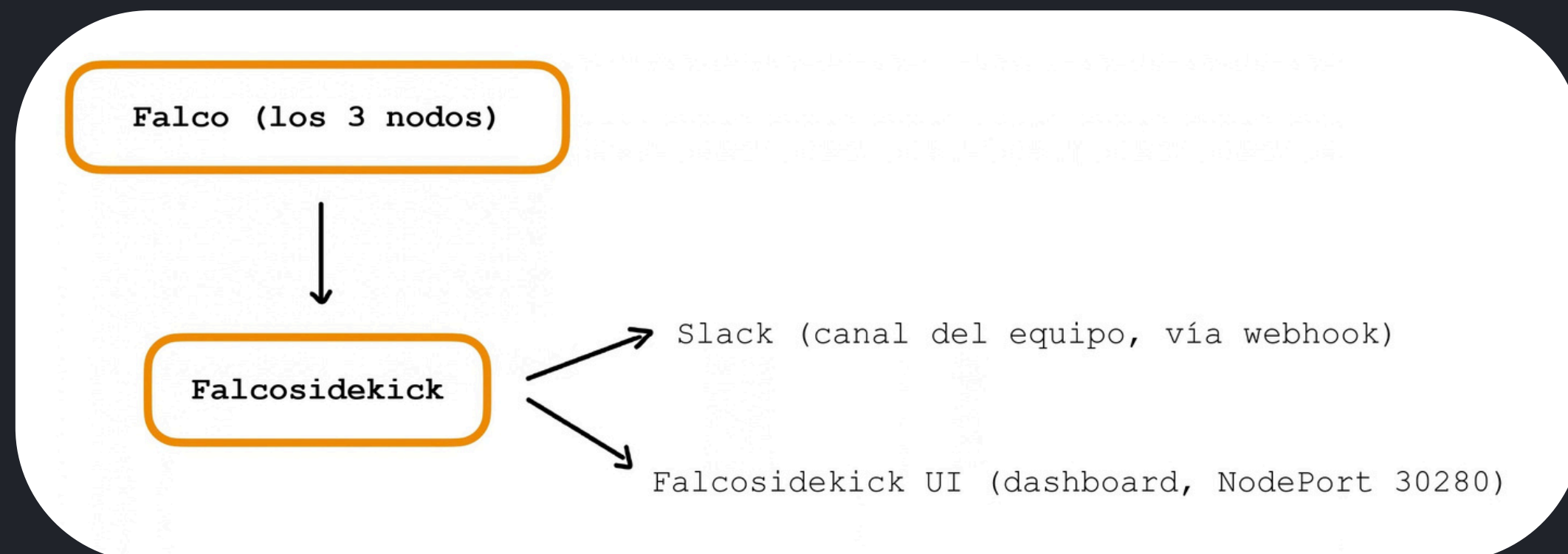
}

Falcosidekick: enrutamiento de alertas {

Falcosidekick recibe todas las alertas de Falco y las reenvía a múltiples destinos en paralelo (Slack y su propio dashboard web)

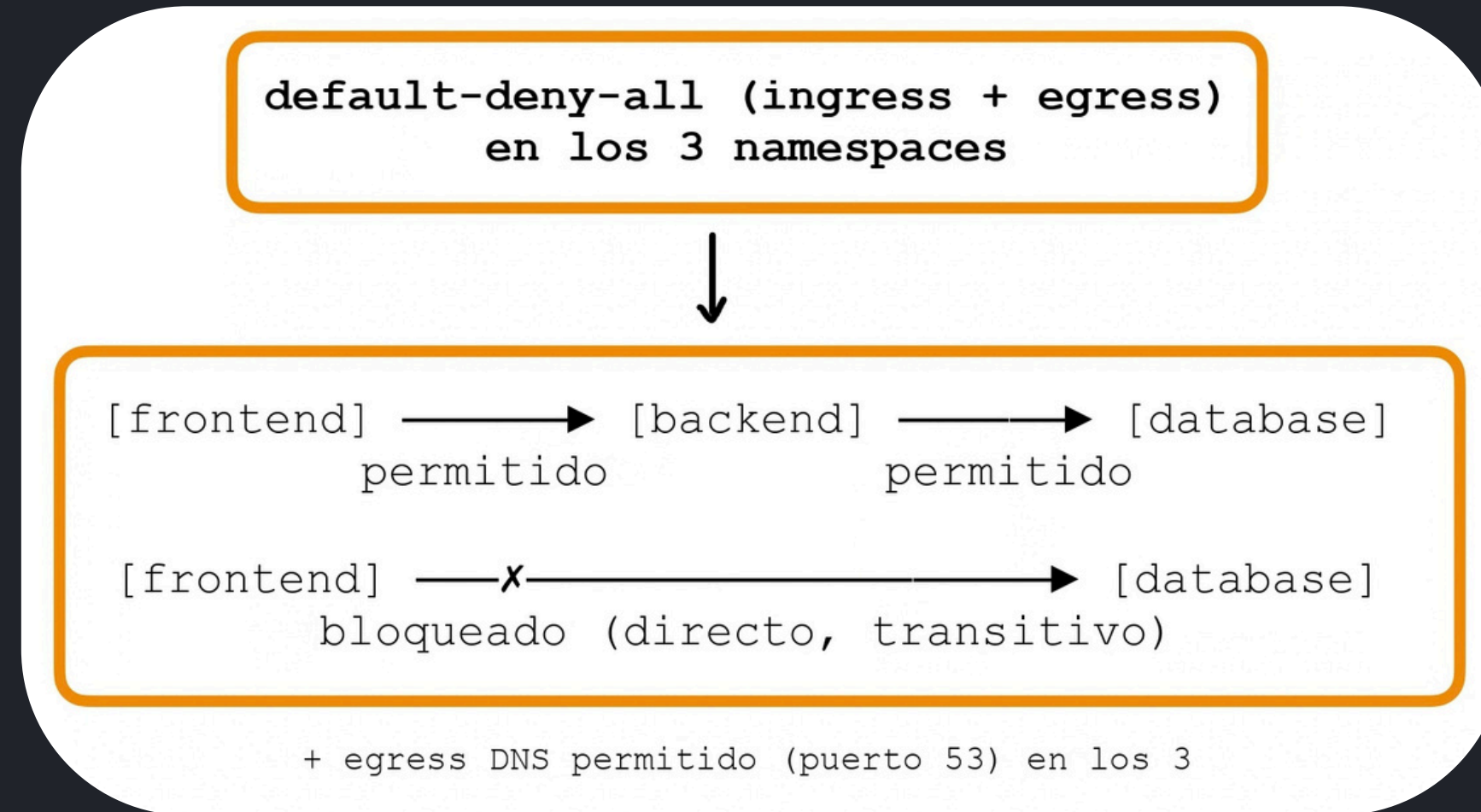
Filtro de severidad: ``minimumpriority: warning`` – las alertas ``Debug`/`Informational`` no llegan a Slack, manteniendo el canal legible sin perder ninguna de las 21 reglas reales (todas clasificadas en ``WARNING``, ``ERROR`` o ``CRITICAL``)

Webhook de Slack: se crea manualmente en ``api.slack.com/apps`` (Incoming Webhook) este paso de configuración que no se puede automatizar con script



}

Network Policies: diseño de 3 capas {



- Punto de partida: default-deny-all en ingress y egress en los 3 namespaces – todo bloqueado por defecto, se permite solo lo explícito
- Excepción obligatoria: egress hacia CoreDNS (puerto 53 UDP/TCP) – sin esto, ningún nombre *.svc.cluster.local resuelve, aunque el tráfico de aplicación sí estuviera permitido
- Whitelists por capa, usando la etiqueta automática kubernetes.io/metadata.name:
 - frontend → egress solo hacia backend, puerto 80
 - backend → ingress solo desde frontend; egress solo hacia database, puerto 80
 - database → ingress solo desde backend; sin egress de aplicación
- Resultado: la segmentación es transitiva, no un filtro de IP superficial – frontend no puede llegar a database aunque conozca su IP directamente
- Manifiestos estándar de Kubernetes (NetworkPolicy), no específicos de Cilium – funcionan igual con cualquier CNI compatible

}

Mapeo a ISO/IEC 27001 {

Control	Qué exige	Evidencia en el proyecto
A.8.16 – Monitoreo de actividades	Monitorear redes/sistemas/apps en busca de comportamiento anómalo	Falco en los 3 nodos + Falcosidekick (Slack/UI) + Hubble (tráfico de red)
A.8.22 – Segregación de redes	Segregar grupos de servicios/usuarios en la red	3 namespaces aislados (frontend/backend/database) con Network Policies
A.8.20 – Seguridad de las redes	Asegurar y controlar redes y dispositivos de red	default-deny con Network Policies. La única excepción es el egress hacia CoreDNS. Solo las redes de pods y de servicios están en la zona trusted. Todo lo demás se rechaza.
A.8.7 – Protección contra malware	Implementar protección contra malware	21 reglas de Falco detectando comportamiento malicioso en tiempo de ejecución
A.5.25 – Evaluación de eventos de seguridad	Decidir qué eventos escalan a incidente	Filtro minimumpriority: warning + exclusión de namespaces de infraestructura en las reglas

}

Conclusion {

Este proyecto mostró que la seguridad en Kubernetes no se resuelve con una sola capa: Falco aporta ojos dentro de cada contenedor, detectando comportamiento malicioso en el momento en que ocurre, mientras que las Network Policies contienen el daño limitando. Lo más interesante de combinarlas es que son independientes entre sí y juntas cubren tanto la detección como la contención, dos mitades del mismo problema que rara vez se atienden con la misma herramienta. Con 21 reglas activas y una segmentación de tres capas verificada en vivo, el clúster queda hoy considerablemente más robustecido que con la configuración por defecto de Kubernetes, y el mismo enfoque escala sin fricción a más microservicios, más namespaces o un entorno productivo real.

}

```
<!--Diplomado Infraestructura en TI - Modulo 4-->
```

Gracias {

}